

InvestigaWarma - MANUAL_TECNICO

Manual Técnico — InvestigaWarma

- 1. Stack tecnológico
- 2. Arquitectura
- 3. Módulos y responsabilidades
- 4. ViewModels
- 5. Room: entidades y DAOs
- 6. Reglas de negocio implementadas en código
- 7. Permisos
- 8. Compilación
- 9. Pruebas
- 10. Mantenimiento y ampliaciones futuras

Manual Técnico — InvestigaWarma

1. Stack tecnológico

Componente	Versión
Kotlin	1.9.24
Android Gradle Plugin (AGP)	8.3.2
Gradle Wrapper	8.6
KSP	1.9.24-1.0.20
Jetpack Compose BOM	2024.06.00
Compose Compiler extension	1.5.14
Material 3	1.2.1
Navigation Compose	2.7.7
Room	2.6.1
kotlinx-coroutines	1.8.1
kotlinx-serialization-json	1.6.3
JDK objetivo	17
compileSdk / targetSdk	34
minSdk	24

Todas las versiones son fijas (sin + ni latest), verificadas manualmente para ser mutuamente compatibles a la fecha de redacción (Kotlin 1.9.24 + AGP 8.3.2 + KSP 1.9.24-1.0.20 + Compose Compiler 1.5.14 es una combinación estable documentada por JetBrains/Google).

2. Arquitectura

MVVM + Repository Pattern, con inyección de dependencias manual (AppContainer), sin frameworks de DI adicionales. Sin backend: toda la persistencia es local (Room).

```
app/src/main/java/com/investigawarma/app/  
├─ AppContainer.kt          Contenedor de dependencias manual  
├─ InvestigaWarmaApp.kt    Application: crea AppContainer y  
                           siembra la BD
```

└─ MainActivity.kt	Host de Compose + NavGraph
└─ data/	
└─ local/	
└─ entity/	15 entidades Room
└─ dao/	15 DAOs
└─ converters/	Converters.kt (List<String>, Map<String,Float>)
└─ seed/	Datos semilla generados (SeedMissions.kt, etc.)
└─ AppDatabase.kt	
└─ DatabaseSeeder.kt	
└─ repository/	PlayerRepository, MissionRepository, ExperimentRepository, ChallengeRepository, CollectionRepository, JournalRepository, StatsRepository
└─ domain/	
└─ model/	Enums, payloads serializables, BadgeKeys
└─ logic/	LevelCalculator, HypothesisValidator, PlantSimulatorEngine, TemperatureSimulatorEngine, ChallengeEvaluator, BadgeRules
(100% JVM puro)	
└─ ui/	
└─ navigation/	Routes.kt, NavGraph.kt
└─ screens/	Onboarding, Home, Zone, Mission (+extras), Journal,
└─ components/	Museum, Stats, Settings, Splash
ProgressCard, EmptyState,	PrimaryButton, AppCard,
ConfirmationDialog, IrisGuide,	SectionHeader,
MagnifierLogo	ZoneIllustrations, AvatarIcon,
└─ theme/	Color.kt, Type.kt, Theme.kt
└─ viewmodel/	ViewModelFactory + 8 ViewModels
└─ util/	VoiceRecorderManager,
SoundHelper, HapticsHelper	

3. Módulos y responsabilidades

- **AppContainer:** crea y expone instancias únicas (singleton de proceso) de AppDatabase, DatabaseSeeder y todos los repositorios/helpers. ViewModelFactory construye los ViewModels a partir de AppContainer.
- **DatabaseSeeder:** puebla la base de datos vacía la primera vez que se abre la app (comprobando `scientificMissionDao().count() == 0`), en una única pasada. Los datos provienen de `data/local/seed/*.kt`, generados por `tools/generate_seed_data.py` para mantener el volumen (40 misiones, 240 pasos, 40 descubrimientos, 30 experimentos, 90 parámetros, 50 desafíos, 20 coleccionables, 15 insignias) sin miles de líneas escritas a mano.
- **Repositorios:** única puerta de acceso a Room desde ViewModels; contienen las transacciones (`MissionRepository.completeMission` usa `db.withTransaction`) y la orquestación entre entidades relacionadas (p. ej. `CollectionRepository.refreshBadges()` recalcula insignias y coleccionables tras cada acción relevante).
- **domain/logic:** lógica de negocio pura, sin Android ni Room, 100% testeable en JVM. Aquí viven las fórmulas de los tres simuladores, el validador de hipótesis, el cálculo de nivel y las reglas de desbloqueo de insignias.

4. ViewModels

ViewModel	Responsabilidad
OnboardingViewModel	Crea el perfil inicial (alias + avatar)
HomeViewModel	Estado del mapa: perfil, nivel, resumen de las 6 zonas
ZoneViewModel	Misiones y desafíos de una zona
MissionViewModel	Máquina de estados del flujo de 6 pasos de una misión
JournalViewModel	Notas de texto/voz, grabación real con VoiceRecorderManager
MuseumViewModel	Colección e insignias
StatsViewModel	Estadísticas calculadas desde StatsRepository
SettingsViewModel	Sonido, vibración, reinicio de progreso

5. Room: entidades y DAOs

Ver docs/BASE_DE_DATOS.md para el detalle completo de tablas, campos, claves e índices. Puntos clave:

- 15 entidades reales, con @ForeignKey y @Index en las relaciones (p. ej. mission_step.missionId → scientific_mission.id, con onDelete = CASCADE).
- Converters.kt traduce List<String> y Map<String, Float> a JSON mediante kotlinx.serialization, registrados con @TypeConverters(Converters::class) en AppDatabase.
- Las escrituras que afectan a varias tablas (completar misión, sembrar la base de datos) usan @Transaction / db.withTransaction { }.

6. Reglas de negocio implementadas en código

- LevelCalculator: 4 niveles con umbrales de XP (0, 300, 800, 1600).
- HypothesisValidator: longitud mínima/máxima por campo, variable ≠ resultado.
- PlantSimulatorEngine / MovementSimulatorEngine / TemperatureSimulatorEngine: fórmulas reales (ver comentarios en el código fuente) que producen un resultado distinto según las variables elegidas.
- ChallengeEvaluator: evalúa las 5 mecánicas de desafío contra el payload persistido en ChallengeEntity.dataJson.
- BadgeRules: evalúa un ProgressSnapshot (contadores reales derivados de la BD) contra las 15 condiciones de insignia.

7. Permisos

Único permiso declarado: RECORD_AUDIO, solicitado en tiempo de ejecución únicamente al intentar grabar una nota de voz (JournalScreen, ActivityResultContracts.RequestPermission). No se declara INTERNET. network_security_config.xml bloquea explícitamente el tráfico en texto claro como medida adicional.

8. Compilación

```
./gradlew clean
./gradlew testDebugUnitTest
./gradlew lintDebug
./gradlew assembleDebug
```

Nota importante: en el entorno donde se generó este proyecto no había Android SDK instalado ni acceso de red a google() / mavenCentral() / Gradle Plugin Portal (confirmado con curl y con un intento real de gradle tasks, ver docs/BUILD_REPORT.md y build_logs/gradle_tasks_attempt.log). Por lo tanto, estos comandos no se pudieron ejecutar con éxito localmente. El repositorio incluye .github/workflows/android-build.yml, que ejecuta exactamente esta secuencia con acceso real a Internet en GitHub Actions.

9. Pruebas

76 pruebas en app/src/test/java/com/investigawarma/app/:

- domain/LevelCalculatorTest.kt, HypothesisValidatorTest.kt, PlantSimulatorEngineTest.kt, MovementSimulatorEngineTest.kt, TemperatureSimulatorEngineTest.kt, ChallengeEvaluatorTest.kt, BadgeRulesTest.kt: 69 pruebas JVM puras, sin Android ni Robolectric.
- data/DatabaseSeederTest.kt: 7 pruebas con Room en memoria + Robolectric (robolectric.properties con sdk=34), verifican cantidades del seed y su idempotencia.

10. Mantenimiento y ampliaciones futuras

- Para añadir contenido (misiones, experimentos, desafíos, coleccionables, insignias), edita tools/generate_seed_data.py y vuelve a ejecutarlo (python3 tools/generate_seed_data.py): regenera los archivos en data/local/seed/ de forma determinista.
- Para añadir un nuevo tipo de simulador, crea un motor puro en domain/logic/, un SimulatorType nuevo en Enums.kt, y conéctalo en ExperimentRepository.runExperiment y en SimulatorContent (UI).
- Para añadir un nuevo tipo de desafío, añade el ChallengeType, un payload serializable en ChallengePayloads.kt, la evaluación en ChallengeEvaluator, y la UI en MissionChallengeContent.kt.
- La versión de base de datos (AppDatabase.version) debe incrementarse y se debe añadir una Migration si se cambia el esquema en una futura versión publicada (v1.0.0 no requiere migraciones al ser la primera versión).